# Sprint 15 · Reto «El mejor modelo, demostrado»

**Esta es la ficha completa del reto, explicada paso a paso** para quien nunca ha trabajado con esta metodología ni con IA. Léela entera **antes de empezar** y vuelve a ella cuando tengas dudas.

- Si es tu primer reto, lee antes Cómo se trabaja un reto.
- Volver al README del sprint · Guía del sprint en PDF · versión ecofriendly para imprimir.
- **Este reto en PDF:** a color · ecofriendly, para imprimir.

---

RETO **Ficha del reto**

## 1 En pocas palabras (léelo primero)

**Qué os pide el cliente.** El cliente (el docente) quiere saber **qué modelo recomendáis y por qué**. No le basta con «el que tiene mejor número». Quiere saber:

- si **la mejora frente al baseline justifica el coste y la complejidad**,
- si el modelo **generaliza** (funciona con datos que no ha visto),
- y que vuestros resultados **se pueden repetir**.

**Este sprint es el corazón técnico del proyecto.** Es donde probáis modelos. Pero lo que se valora **no es probar muchos**, sino **probar con orden, anotar todo y elegir con honestidad**.

**Qué vais a hacer, en cinco pasos:**

1. **Preparar el terreno:** semillas fijas, estructura del registro de experimentos, y repetir el baseline dentro del registro.
2. **Probar al menos 3 modelos** (clásicos) y **al menos uno neuronal** (o LLM/RAG si vuestro brief es el 3), con la misma partición.
3. **Anotar cada prueba** en `experimentos/registro.csv` (plantilla; cuaderno de ayuda `S15_01_seguimiento_experimentos`) (≥ 12 filas, **de todas las personas**).
4. **Elegir el modelo candidato** comparando con media ± desviación, **sin tocar el test**.
5. **Evaluar en el test una sola vez** y preparar una primera explicación del modelo.

**Qué vais a entregar:** el registro de experimentos, el notebook `03_modelos.ipynb`, la justificación de 1 página del candidato, el modelo guardado y una primera explicación (importancia de variables, SHAP o Grad-CAM).

**Qué vais a aprender haciéndolo:** a **experimentar con método**, a **evitar la fuga de datos** y a **argumentar una elección** con evidencias y no con corazonadas.

**Cuánto tiempo:** tres jornadas.

## 2 Palabras que van a aparecer en este reto

| Palabra | Qué significa, en sencillo |
|---|---|
| **Registro de experimentos** | Una tabla donde se anota cada prueba: qué modelo, qué ajustes, qué resultado, quién y cuándo |
| **Modelo candidato** | El modelo que proponéis como mejor opción |
| **Hiperparámetro** | Un ajuste del modelo que se fija antes de entrenar |
| **Media ± desviación** | Resultado promedio de varias evaluaciones y cuánto varía |
| **Validación cruzada** | Evaluar varias veces con particiones distintas del entrenamiento |
| **Fuga de datos** | Usar, al entrenar o elegir, información que no estaría disponible al decidir |
| **Pipeline** | Cadena de preparación + modelo que se ajusta solo con datos de entrenamiento |
| **Sobreajuste** | El modelo memoriza el entrenamiento y falla con datos nuevos |
| **Explicabilidad** | Poder decir por qué el modelo decide lo que decide |
| **SHAP / Grad-CAM** | Técnicas para ver qué variables (SHAP) o qué zonas de una imagen (Grad-CAM) influyen |
| **Coste / beneficio** | Qué se gana (mejor métrica) frente a qué cuesta (tiempo, complejidad, falta de explicación) |

| Palabra | Qué significa, en sencillo |
|---|---|
| **Ruido** | Variación que ocurre por casualidad. Una diferencia pequeña puede ser solo ruido |

## 3 El cliente

El cliente (el docente) quiere saber **qué modelo recomendáis y por qué**. No le basta con «el que tiene mejor número»: quiere saber si la mejora frente al baseline justifica el coste y la complejidad, si el modelo generaliza, y que vuestros resultados se pueden repetir.

## 4 Qué se entrega

Al cierre del sprint (3/12), en el repositorio:

1. **Al menos 3 modelos** comparados con el **baseline**, con validación correcta (sin fuga; partición temporal si hay series) y **al menos uno neuronal** (o LLM/RAG en el brief 3).
2. **Registro de experimentos** (`experimentos/registro.csv`) con todas las pruebas: ≥ 12 filas recomendadas, con **filas de todas las personas** del equipo.
3. **Modelo candidato** elegido con una justificación de 1 página: tabla comparativa, coste/beneficio (tiempo, complejidad, explicabilidad) y por qué gana.
4. Notebook `notebooks/03_modelos.ipynb` ejecutable de arriba abajo con semillas fijas.
5. Primera **explicación del modelo** (importancia, SHAP o Grad-CAM según el caso) para la evaluación del Sprint 16.

## 5 Restricciones

- El **test se usa una sola vez**, con el modelo ya elegido. Elegir por el test invalida el resultado.
- **Semillas fijas** y versiones anotadas. Sin claves de API ni datos personales en el repositorio.
- Si el cómputo es limitado (GPU de Colab, API de pago), usad subconjuntos y anotad el coste en el registro.
- Nada de «probar 200 combinaciones y quedarse con la mejor»: definid antes qué vais a probar y por qué.

## 6 Criterios de evaluación

Rúbrica del proyecto (100 puntos, sección 5 de los briefs). En este sprint pesan sobre todo:

| Criterio | Qué se mira en el Sprint 15 |
|---|---|
| **C · Modelado y experimentación (20)** | Baseline y ≥ 3 modelos; validación correcta; ajuste de hiperparámetros; **registro de experimentos**; control de fuga; justificación coste/beneficio y **explicación del porqué** de los resultados (nivel «Excelente») |
| **D · Evaluación (10)** | Primeras curvas y análisis de errores |
| **F · Reproducibilidad (10)** | Semillas, versiones y ejecución de arriba abajo |
| **G · Equipo (10)** | Roles rotados (roles nuevos este sprint), commits repartidos, retro con acción del sprint anterior |

## 7 Pasos sugeridos

| Cuándo | Paso |
|---|---|
| Día 1 | Fijar semillas y estructura del registro. Repetir el baseline con el registro. Entrenar el primer modelo clásico y anotar. |
| Día 2 | Segundo y tercer modelo; red neuronal (o LLM/RAG); ajustar 1-2 hiperparámetros clave. Tutoría de 15 min: qué comparación estáis haciendo y qué fuga podría haber. |
| Día 3 | Comparativa final con media ± desviación; **elegir el candidato** (sin tocar el test hasta este momento); evaluar en test **una vez**; preparar la review. |

## 8 Guía paso a paso para principiantes

**Paso 1 · Preparar el terreno (día 1)**

1. **Fijad una semilla** (por ejemplo, 42) y anotad las versiones de librerías.
2. **Cread `experimentos/registro.csv`** (partid de la plantilla) con estas columnas (o similares): *id · fecha · persona · modelo · hiperparámetros · partición · métrica_validación · desviación · tiempo · notas*.
3. **Repetid el baseline y anotadlo como primera fila del registro.**
4. **Definid la partición** que usará todo el equipo (temporal si hay series; por grupos si hay fotos del mismo objeto). **No la cambiéis.**
5. **Definid antes qué vais a probar y por qué** (por ejemplo, regresión logística, bosque, boosting y una red). No es válido «probar 200 combinaciones y quedarse con la mejor».

**Paso 2 · Modelos clásicos y neuronal (día 2)**

1. Entrenad **al menos 3 modelos** con la misma partición y **anotad cada prueba**.
2. Entrenad **al menos un modelo neuronal** (o el RAG/LLM en el brief 3).
3. Ajustad **1–2 hiperparámetros clave** de cada familia, solo con validación.
4. Cada persona debe **anotar al menos una fila** en el registro.
5. **Tutoría de 15 minutos con el docente:** contad qué comparación hacéis y qué fuga podría haber.

**Paso 3 · Comprobar que no hay fugas de datos (día 2, antes de elegir)**

Pasad esta lista (también está al final de la ficha del sprint):

- ¿Las variables se conocerían **en el momento de decidir**?
- ¿El escalado, la imputación y la codificación se aprenden **solo con el entrenamiento** (dentro de un `Pipeline`)?
- ¿La partición respeta el **tiempo** o los **grupos**?
- ¿Se ha ajustado algo mirando el test? Si sí, hay que rehacer la evaluación.
- ¿La métrica es **demasiado buena para ser verdad**? Sospechad.

**Paso 4 · Elegir el candidato (día 3, mañana)**

1. Calculad **media ± desviación** (o intervalo) de cada modelo.
2. Comparad **teniendo en cuenta la incertidumbre**: si dos modelos se solapan, la diferencia puede ser ruido.
3. Añadid **coste y complejidad**: tiempo de entrenamiento, tamaño, facilidad de explicación.
4. **Elegid el candidato** y escribid la justificación de 1 página:

| Modelo | Métrica (media ± desv.) | Tiempo | Explicabilidad | Comentario |
|---|---|---|---|---|
| Baseline | | | | |
| Modelo A | | | | |
| Modelo B | | | | |
| Neuronal | | | | |

5. **Hipótesis sobre por qué gana ese modelo** (nivel «Excelente» de la rúbrica): por ejemplo, «los datos tienen relaciones no lineales entre antelación y canal que un árbol captura mejor que una regresión».

**Paso 5 · Test una sola vez y explicación (día 3)**

1. **Solo ahora** evaluad el candidato en el test, **una única vez**. Si el resultado no os gusta, **no volváis a elegir con él**: anotadlo y explicadlo.
2. **Guardad el modelo** en `modelo/` con un `metadatos.json` (versión, fecha, métricas, variables).
3. **Primera explicación del modelo:** importancia de variables, SHAP (tabular) o Grad-CAM (imágenes). Se profundizará en el Sprint 16.
4. Preparad la review de 5 minutos.

## 9 Cómo usar la IA en este reto

| Para qué | Ejemplo de petición | Qué tenéis que comprobar |
|---|---|---|
| Entender un modelo | «Explícame cuándo conviene un modelo de boosting frente a una regresión logística, con un ejemplo sencillo» | Que la explicación encaja con **vuestros datos** |

| Para qué | Ejemplo de petición | Qué tenéis que comprobar |
|---|---|---|
| Depurar un error | «Este código de scikit-learn da este error: *[error]*. Datos: *[estructura]*» | Que entendéis la causa |
| Detectar fugas | «Aquí está mi pipeline: *[código]*. ¿Hay alguna operación que use el test o el futuro?» | Que **verificáis** la respuesta; la IA puede pasarlas por alto o ver fugas inexistentes |
| Interpretar SHAP | «Tengo este gráfico SHAP (descripción). ¿Qué significa que la variable X tenga ese efecto?» | Contrastadlo con el sentido del negocio |

**Reglas del reto:**

- **El test se usa una sola vez, con el modelo ya elegido.** Elegir por el test invalida el resultado.
- **Semillas fijas** y versiones anotadas. **Sin claves ni datos personales** en el repositorio.
- Si el cómputo es limitado (GPU de Colab, API de pago), usad subconjuntos y **anotad el coste** en el registro.
- Declarad el uso de IA. En el registro, **marcad las pruebas hechas con código generado por IA**.

## 10 Errores frecuentes (y cómo evitarlos)

| Error | Por qué ocurre | Cómo evitarlo |
|---|---|---|
| Elegir el modelo mirando el test | «Para ver cuál es mejor» | Elegid solo con validación; el test, una vez |
| Resultados «increíbles» (99 %) | Fuga de datos | Revisad variables y partición con el checklist |
| Registro con 3 filas de una sola persona | El responsable técnico lo hace todo | Cada persona prueba algo y lo anota |
| Comparar con particiones distintas | Cada uno prueba por su lado | Una partición común, guardada |
| Mejor número = mejor modelo | Se mira solo la métrica | Añadid coste, simplicidad y explicabilidad |
| Mezclar fotos del mismo objeto entre train y test | En imágenes es fácil | Particionad por objeto o por parcela |

## 11 Un buen resultado se parece a...

> *«Elegimos el boosting (F1 0,71 ± 0,02) frente al bosque (0,70 ± 0,02) y a la red (0,72 ± 0,04). La red es 0,01 mejor pero con una desviación doble y 15 veces más lenta: la diferencia puede ser ruido. El boosting gana frente a la regresión porque captura interacciones entre antelación y canal. Evaluado una sola vez en test: F1 0,69.»* (Cifras de ejemplo.)

Un **registro de experimentos** correcto tiene ≥ 12 filas, de todas las personas, con hiperparámetros, métricas y notas («empeoró al aumentar profundidad: sobreajuste»).

## 12 Antes de entregar: autoevaluación rápida

Repasad estas preguntas y rellenad después la hoja autoevaluacion_y_coevaluacion.md (cada persona sube la suya con su entrega).

| Pregunta | Sí / No |
|---|---|
| ¿Hay baseline + ≥ 3 modelos + 1 neuronal (o LLM/RAG) con la **misma** partición? | |
| ¿La validación no tiene fuga (temporal en series; sin mezclar fotos del mismo objeto)? | |
| ¿`registro.csv` tiene todas las pruebas y al menos una fila por persona? | |
| ¿Semilla y versiones anotadas; el notebook se ejecuta de arriba abajo? | |
| ¿El candidato está elegido con justificación (rendimiento, coste, explicabilidad)? | |
| ¿El test se evaluó una sola vez, después de elegir? | |
| ¿Hay modelo guardado y primera explicación? | |
| ¿Roles rotados, tablero actualizado y retro con acción concreta? | |

## 13 Si os atascáis

- **No sabemos qué modelos probar:** repasad el leaderboard del Sprint 8 y la red del Sprint 10; usad las mismas familias.
- **Los resultados de dos modelos son casi iguales:** perfecto; elegid el más simple y explicadlo.
- **No tenemos GPU o es lenta:** usad un subconjunto de datos y anotadlo en el registro.
- Más ayuda: Cómo se trabaja un reto.

## 14 Qué se enseña en la review (3/12)

Cada equipo enseña en 5 minutos: (1) la tabla de modelos con media ± desviación, (2) el candidato elegido y **por qué** (no solo la métrica), (3) una fuga que evitó o detectó, (4) una hipótesis sobre por qué gana ese modelo, (5) su registro de experimentos. El resto de equipos pregunta: «*¿cómo sabéis que la diferencia no es ruido?*»

## 15 Definition of Done (lista de comprobación)

Si queda una casilla sin marcar, **el reto no está terminado**.

- [ ] Baseline + ≥ 3 modelos + 1 neuronal (o LLM/RAG) comparados con la **misma** partición.
- [ ] Validación sin fuga; en series, partición temporal; en imágenes, sin mezclar fotos del mismo objeto entre train y test.
- [ ] `experimentos/registro.csv` con todas las pruebas y **al menos una fila por persona**.
- [ ] Semilla fija y versiones anotadas; el notebook se ejecuta de arriba abajo.
- [ ] Modelo candidato elegido con justificación (rendimiento, coste, simplicidad, explicabilidad).
- [ ] Test evaluado **una sola vez**, después de elegir.
- [ ] Modelo guardado (`modelo/`) con `metadatos.json` si es posible.
- [ ] Roles rotados, tablero actualizado y retro con acción concreta.
- [ ] Autoevaluación y coevaluación rellenadas (cada persona sube su hoja).

## 16 Checklist rápido de validación sin fuga

- ¿Las variables se conocerían **en el momento de decidir**?
- ¿El escalado, la imputación y la codificación se aprenden **solo con el entrenamiento** (dentro de un `Pipeline`)?
- ¿La partición respeta el **tiempo** o los **grupos** (usuarios, parcelas, imágenes del mismo objeto)?
- ¿Se ha ajustado algo mirando el test? Si sí, hay que rehacer la evaluación con otro test o con validación anidada.
- ¿La métrica es demasiado buena para ser verdad? Sospechad.